VPC Peering, Transit Gateway, and Endpoints

Explore how to connect VPCs using VPC peering for simple point-to-point links and transit gateways for scalable hub-and-spoke architectures. Understand how VPC endpoints keep AWS service traffic private by avoiding public internet exposure. This lesson helps you design secure, scalable network architectures meeting AWS best practices.

VPCs are designed to be isolated. Imagine the development team has one VPC and the data science team has another, but the developers need to connect to an RDS database in the data science VPC. How should this be handled?

There are two options:

1. **VPC peering:** A lightweight, direct connection between two VPCs.
2. **Transit Gateway:** A scalable hub-and-spoke model connecting many VPCs and on-premises networks.

The answer depends on a few questions:

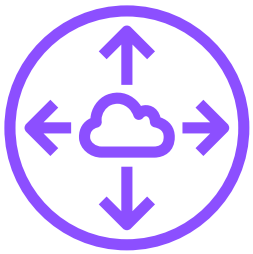
- How many VPCs need to be connected?
- Whether they span Regions or accounts?
- The direction and frequency of the traffic?

VPC peering: Point-to-point simplicity

Ask

VPC peering is a feature that allows us to connect different VPCs securely, that their resources communicate with each other as if they reside in the same VPC. It uses the AWS global network to route traffic between VPCs. Because all

communication happens through private IP addresses, VPC peering connections never go outside the private IP space and use the internet.



It is important to note that VPC peering requires no additional physical hardware because it uses AWS's existing infrastructure for connection. Therefore, no single point of failure or bandwidth bottleneck occurs in the communication.

But here's the catch: it's non-transitive.

> **Note:** If VPC A is peered with VPC B, and B is peered with C, A cannot talk to C.

VPC peering in 3 VPCs

Ask

This makes it simple, but not scalable for environments with more than a few VPCs. To set up peering:

- The two VPCs must have non-overlapping CIDR blocks.
- We must update route tables in both VPCs to send traffic through the peering connection.
- Security groups still apply; traffic can be blocked if SGs don't allow it.

VPC peering in AWS Regions

VPC peering allows us to connect VPCs in the same and different AWS Regions.

- **Intra-region connection:** When configuring a VPC peering connection between VPCs within the same region, we can use security groups to allow traffic to and from the security group of the peering VPCs.
- **Inter-region connection:** If we connect the VPCs from different regions, we must use the CIDR address of the connecting VPC as the source or destination in the security group to control traffic. Resources use private IP addresses to communicate with each other and never use public IP addresses. Therefore, the data transferred over the network is encrypted.

We can establish VPC peering. between two VPCs using the AWS CLI command given below:

main.sh C++

```
aws ec2 create-vpc-peering-connection \
  --vpc-id vpc-11111111 \
  --peer-vpc-id vpc-22222222 \
  --peer-region us-west-2
```

Ask

AWS CLI command to establish VPC peering between two VPCs

Important follow-up:

After creating the peering connection, we must:

- Accept the connection request.
- Update route tables in both VPCs to send the traffic to the appropriate destination.
- Check SG rules to allow traffic between VPCs.

> **Warning:** Peering does *not* support DNS resolution across VPCs by default. We must enable it.

Transit Gateway: Hub-and-spoke simplicity at scale

AWS Transit Gateway is a regional resource used to connect thousands of VPCs and on-premises resources. It acts as a hub-and-spoke in the middle of the VPCs and routes traffic to them. It can manage connections to Direct Connect, Site-to-Site VPNs, and even a custom gateway.

Transit Gateways use attachments to link VPCs. You can think of the Transit Gateway as the router and each attachment as the port where a VPC conn exchange traffic. To connect VPCs using Transit Gateways, we first create the

Ask

Transit Gateway. The AWS CLI command given below creates the Transit Gateway:

main.sh Shell

```
aws ec2 create-transit-gateway \
  --description "My Dev Transit Gateway" \
  --options AmazonSideAsn=64512 \
  --tag-specifications 'ResourceType=transit-gateway,Tags=[{Key=Name,\
```

AWS CLI command to create a Transit Gateway

Then, to connect our VPC, we create a Transit Gateway attachment for our VPC using the AWS CLI command given below:

main.sh Shell-20

```
aws ec2 create-transit-gateway-vpc-attachment \
  --transit-gateway-id tgw-12345678 \
  --vpc-id vpc-0abc123456 \
  --subnet-ids subnet-1234abcd subnet-5678efgh
```

Ask

AWS CLI command to create a Transit Gateway attachment

Once the attachment is created, we update the route tables of our Transit Gateway and VPC to route the traffic to the appropriate CIDR destinations.

Connecting VPCs using Transit Gateway

VPC endpoints: Keeping AWS traffic privat

Ask

So far, we've discussed VPC-to-VPC connectivity. But what about the connection between VPC and the AWS services lying outside the VPC? For example, an EC2

instance in a private subnet wants to store a file in an S3 bucket. One way to establish this connection could be through a NAT gateway. A **NAT gateway** enables instances in a private subnet to initiate outbound connections to the Internet while preventing inbound traffic from the internet.

EC2 instance in private subnet connecting to the S3 bucket

However, this solution has a downside. The data on the private EC2 instance would traverse the public internet. To address this issue, AWS offers VPC endpoints.

A **VPC endpoint** is a feature in AWS that allows our AWS resources within the VPC to privately connect to supported AWS services outside the VPC without traversing the public Internet.

Think of it as a private tunnel from our VPC to an AWS service, where traffic stays entirely within the AWS network. Eliminating the need for NAT gateways, internet gateways, or public IPs improves security, reduces latency, and often lowers cost.

Ask

Some of the VPC endpoints use **AWS PrivateLink** to establish the secure channel. This VPC-based service creates a private connection between VPCs, AWS services, and on-premises resources in an AWS Region.

Using PrivateLink, we can connect services in one AWS account, different accounts, and AWS partner-supported services. It uses the AWS private network for communication, so the traffic never goes to the public internet. We don't need a NAT gateway, IGW, or AWS Direct Connect for connection to the resources in a private subnet.

Types of VPC endpoints

VPC endpoints are the resources providing the pathway to access the services. There are three types of VPC endpoints, two of which are powered by AWS PrivateLink. The table below shows their main points:

Interface Endpoint (PrivateLink)	Gateway Endpoint	Gateway L
Uses an elastic network interface with a private IP address.	An endpoint that is used as a target in the route table for the AWS service.	Used to conn services conf L
It works as an entry point to an AWS service, a service hosted by another AWS account, or a supported AWS partner service.	It doesn't use IP addresses, PrivateLink, or Security Groups.	It uses an ir inte

Ask

It's a regional resource that can be used only in the region it is created in.	It only supports AWS S3 and AWS DynamoDB and allows IPv4 TCP traffic.	The GWLB e subnet othe service consu th
It allows us to choose a subnet per availability zone, attach a security group, and write an endpoint policy. Attaching a security group is a must; if we don't specify a security group, the default one is attached automatically.	Gateway endpoints can only connect to VPCs and resources in the same region.	All the incom routed throu according to in
It supports most of the AWS services and allows IPv4 TCP traffic only.	Allows to define endpoint policy.	

Private access to AWS services using VPC endpoints: A real-world scenario

To understand the potential of VPC endpoints, let's consider a use case scenario. Imagine we've deployed a backend application on EC2 instances inside a private subnet. This application needs to:

- Store image files in Amazon S3.
- Query DynamoDB for user session data.
- Retrieve API keys from AWS Secrets Manager.

Now, because these EC2 instances are in a private subnet, they don't have the internet access by default, which raises a challenge: how do we connect to AWS services like S3 or Secrets Manager without sending traffic over the public internet?

Ask

One common solution is to route outbound traffic through a NAT gateway sitting in a public subnet. However, as discussed previously, this design exposes our

traffic to the internet.

Imfrastrcuture to connect to AWS service to resources in a private subnet

Therefore, we use VPC Endpoints to allow our private resources, like EC2, Lambda, or Fargate, to connect directly to AWS services from within the VPC, using the AWS global backbone instead of the public internet.

To enable communication in this scenario, we'll configure a gateway endpoint for S3 and another for DynamoDB, each pointing to the appropriate route table used by your private subnet. Then, for Secrets Manager, we'll set up an interface endpoint, assigning it to a subnet and security group.

Ask

Additionally, we'll enable private DNS, so the application continues using the standard `secretsmanager.us-east-1.amazonaws.com` endpoint without code changes. Now, when the app uploads a file to S3 or retrieves a secret, that traffic never touches the Internet. It stays inside AWS, uses private IPs, and travels securely.

Imfrastrcuture to connect to AWS service to resources in private subnet

Compared to using NAT gateways, this architecture is more secure, cost-efficient, and better aligned with data privacy requirements. Gateway endpoints for services like S3 and DynamoDB incur no additional cost, and public IPs or NAT infrastructure are no longer required. In enterprise environments, especially those

Ask

subject to financial or government regulations, this approach is not optional—it is mandatory.

Conclusion

VPC peering enables private, point-to-point communication between two VPCs, but it does not scale well because connections are non-transitive. For larger environments, Transit Gateway provides a hub-and-spoke model that supports transitive connectivity and centralized routing across multiple VPCs and hybrid networks.

VPC endpoints enable private access to AWS services such as S3, DynamoDB, and Secrets Manager from within a VPC. Replacing NAT gateways with gateway and interface endpoints removes public internet exposure, reduces cost, and

Ask